THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 25

Photon Engine — What It Actually Accelerates

SECTION 01

What Photon Actually Is

CORE CONCEPT

Photon is the Databricks-native vectorized query engine that accelerates SQL workloads, DataFrame API calls, ETL pipelines, and stateless streaming workloads. It processes data in columnar batches, delivering significant performance improvements over traditional row-based execution, and is fully compatible with Apache Spark APIs, so existing code works with no changes required.

Photon first appeared as a preview in 2020 and went generally available in 2022. It now sits underneath every Databricks SQL warehouse and every Photon-enabled classic or serverless cluster.

SECTION 02

How Photon Fits Inside Spark, Not Instead of It

A common misconception is that Photon replaces Spark. It doesn't. It replaces one specific layer within Spark's execution pipeline.

```
Spark SQL -> Catalyst Optimizer -> Execution Layer -> Cluster HW
             (unchanged)            (Photon replaces this)
```

- Apache Spark's Catalyst optimizer still generates the logical and physical query plan, exactly as it always has.
- For supported operations, Photon replaces the JVM-based Spark SQL execution engine with a native C++ runtime at that point.
- The Spark APIs on top are completely unchanged; the same DataFrame and SQL surface you write against still applies.

SECTION 03

Why a Native C++ Engine Is Faster

Standard Spark execution runs on the Java Virtual Machine (JVM), which introduces overhead like garbage collection pauses and memory management layered on top of the actual computation.

Aspect	Traditional JVM Execution	Photon (Native C++)
Processing Unit	Row by row	Columnar batches
CPU Utilization	Limited SIMD usage	SIMD instructions process many rows per CPU cycle
Overhead	JVM garbage collection, object overhead	Direct hardware-level execution
Typical Speedup	Baseline	3x to 8x average on SQL and ETL

		workloads

SECTION 04

What Photon Does NOT Accelerate

IMPORTANT NUANCE

When Photon hits an unsupported operator, it falls back to the JVM transparently for the duration of that operator, then picks up again on the next supported one. Python UDFs, stateful streaming, and some ML operators commonly fall back to the JVM, meaning a workload can pay the Photon compute rate while only getting partial acceleration.

This is a genuinely important cost consideration: Photon-enabled clusters cost more per DBU, so a workload dominated by unsupported operators may see a worse cost-to-performance ratio than running without Photon at all.

SECTION 05

When Photon Is Enabled

Compute Type	Photon Default
Serverless SQL Warehouse	Always on, has been the default runtime since 2025
Serverless Compute (general)	Default runtime
Classic Clusters	Not on by default; requires an explicit checkbox or API flag

SECTION 06

Vectorized vs Row-Based Execution

```
ROW-BASED (traditional JVM execution)
Row 1 -> process -> Row 2 -> process -> Row 3 -> process -> ...
(one row at a time, JVM overhead per operation)

VECTORIZED (Photon, columnar batches)
[Row 1, Row 2, Row 3, ... Row N] -> process entire batch at once
(SIMD instructions handle many values per CPU cycle)
```

REMEMBER THIS FOR INTERVIEWS

Photon replaces the execution layer, not the planning layer. Catalyst still decides the query plan; Photon decides how fast that plan actually runs, and only for the operators it supports natively.

SECTION 07

Glossary — Day 25 Terms

Term	Definition
Photon	Databricks' native, vectorized C++ query engine that replaces JVM-based Spark SQL execution for supported operators.
Vectorized Execution	Processing data in columnar batches rather than row by row, enabling SIMD-based CPU parallelism.
Catalyst Optimizer	Apache Spark's query planning component, which continues to generate query plans regardless of whether Photon is enabled.
SIMD (Single Instruction, Multiple Data)	A CPU capability that lets one instruction operate on multiple data values simultaneously, key to Photon's speed.
JVM Fallback	The transparent process where Photon hands execution of an unsupported operator back to the JVM, then resumes on the next supported one.

SECTION 08

Interview Questions & Answers

Q1. What is Photon?

A. Databricks' native, vectorized query engine, built in C++, that accelerates SQL, DataFrame, ETL, and stateless streaming workloads by processing data in columnar batches instead of row by row.

Q2. Does Photon replace Apache Spark?

A. No. It replaces the JVM-based execution layer for supported operators. Apache Spark's Catalyst optimizer still generates the query plan; Photon only changes how that plan is executed.

Q3. Why is Photon typically faster than standard JVM-based Spark execution?

A. It processes data in columnar batches using SIMD instructions, which let a single CPU instruction operate on many values at once, avoiding JVM overhead like garbage collection and delivering roughly 3x to 8x speedups on average for SQL and ETL workloads.

Q4. Does Photon accelerate every type of workload?

A. No. Python UDFs, stateful streaming, and some ML operators commonly fall back to the JVM when Photon doesn't support them natively, meaning those parts of a workload run at standard JVM speed even on a Photon-enabled cluster.

Q5. Is Photon enabled by default on all Databricks compute?

A. It has been the default runtime for Serverless SQL warehouses and general serverless compute since 2025. Classic clusters do not enable it by default and require an explicit checkbox or API flag.

Q6. Why might enabling Photon sometimes hurt cost-efficiency?

A. Photon-enabled clusters cost more per DBU. If a workload is dominated by operators that fall back to the JVM anyway, you pay the higher Photon rate while gaining little to no actual acceleration.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab